

Composed Timeout Budgets, Worked Out by Hand

HOW AN END-TO-END BUDGET SPLITS ACROSS LAYERS — AND WHY A STACKED RETRY IS PURE WASTED COMPUTE

What this document does. It takes one concrete request — a *Nexus v7* interactive-class call with a 500ms end-to-end SLO — and shows how that budget B decomposes across the gateway, auth, scheduler, engine, and a streaming margin, and why the sum must stay inside B . It then composes timeouts with *retries* and proves the constraint $(\text{num_retries} + 1) \times \text{per_try_timeout} \leq \text{outer_timeout}$. With a 500ms outer timeout and 250ms per try, a third attempt can never land — it is pure wasted compute. Every number is produced by the reference script (Appendix A, `m07_t04_timeout_budget_engine.py`).

The one idea. Timeouts *compose*. A retry does not get a fresh clock; it spends the same outer budget the first attempt was already spending. If you configure more attempts than the budget can hold, the surplus attempts are dead on arrival — they consume GPU and return nothing.

CONCEPTS COVERED, IN ORDER

the budget decomposition $B = T_{\text{gw}} + T_{\text{auth}} + T_{\text{sched}} + T_{\text{engine}} + T_{\text{margin}}$; the 10% margin rule; the retry/timeout constraint $(\text{num_retries} + 1) \cdot \text{per_try} \leq \text{outer}$; the worked 500/250 case where the third attempt is waste; the maximum useful number of retries; and why stacked timeouts burn compute on requests that are already dead.

INTERNAL LEARNING MATERIAL. NUMBERS ARE REPRODUCIBLE FROM APPENDIX A. v1.0

Contents

1	The system under study	2
2	Step 1 — Decomposing the budget across layers	2
3	Step 2 — Composing timeouts with retries	3
4	Step 3 — The composed-timeout trap	3
5	Step 4 — Why stacked timeouts waste <i>compute</i> , not just time	4
6	The argument, at a glance	5
6.1	Equation cheat-sheet	5
	Appendix A — Reference implementation	5

1 The system under study

An end-to-end LLM request touches many layers in series: gateway $\rightarrow$ auth $\rightarrow$ scheduler $\rightarrow$ engine $\rightarrow$ response streaming. Each layer has its own timeout. The end-to-end (outer) timeout is the user-facing SLO; the per-layer timeouts must fit inside it. We fix one concrete request.

Quantity	Symbol	Value here
End-to-end budget (outer timeout)	B	500 ms
Gateway timeout	T_{gateway}	50 ms
Auth timeout	T_{auth}	30 ms
Scheduler timeout	$T_{\text{scheduler}}$	20 ms
Engine (inference) timeout	T_{engine}	350 ms
Streaming margin	T_{margin}	50 ms
Per-try timeout (retry policy)	—	250 ms

Intuition

Think of B as a fixed-length relay race — 500 ms of track, no more. Each runner (gateway, auth, scheduler, engine) gets a marked segment; the segments must add up to the track, not overshoot it. A *retry* is not a second, longer track — it is sending a fresh runner down the *same* 500 ms of track from the start line. If two full attempts already use the whole track, a third runner has no track left to run on; they are told to stop the moment they start.

2 Step 1 — Decomposing the budget across layers

The per-layer timeouts are a partition of the end-to-end budget. They must sum to B (or less), never more.

Timeout budget decomposition

$$B = T_{\text{gateway}} + T_{\text{auth}} + T_{\text{scheduler}} + T_{\text{engine}} + T_{\text{margin}}.$$

Add up the Nexus allocation

$$50 + 30 + 20 + 350 + 50 = 500 \text{ ms} = B. \quad \checkmark$$

The engine gets the lion's share (350 ms, 70%) because token generation is the expensive part; the supporting layers are deliberately thin. The streaming margin (50 ms) is held in reserve so the response can begin flushing before the hard cutoff.

Mini-example — the 10% margin rule

The course guidance is “ $\sum$ per-layer timeouts $< B$, leave 10% margin.” Here the *hard* layers (everything except the streaming margin) total $50 + 30 + 20 + 350 = 450 \text{ ms} = 90\%$ of B , and the 50 ms margin is the remaining 10%. That slack absorbs scheduling jitter and clock skew so that a request which is *almost* on time is not killed by rounding. Allocate the layers to exactly B with no margin and any jitter at all tips you over.

Watch out

If the per-layer timeouts *summed to more* than B , the outer timeout would still win — the user gets their answer or error at B regardless — but the inner layers would keep working on a request the gateway has already abandoned. The layers waste compute chasing a request that is already dead. The sum-to- B constraint is what keeps every layer honest about the clock it actually has.

3 Step 2 — Composing timeouts with retries

Retries reuse the outer budget. With `num_retries` retries there are `num_retries + 1` total attempts (the initial try plus the retries), each bounded by `per_try_timeout`. For the worst case — every attempt running to its full per-try limit — to fit inside the outer budget:

The retry/timeout constraint

$$(\text{num_retries} + 1) \times \text{per_try_timeout} \leq \text{outer_timeout}.$$

Sweep `num_retries` at `per_try = 250 ms`, `outer = 500 ms`:

num_retries	attempts	worst case (ms)	Fits ≤ 500 ?
0	1	250	✓ fits
1	2	500	✓ fits (exactly)
2	3	750	overflow by 250
3	4	1000	overflow by 500

The largest `num_retries` whose worst case still fits is `num_retries = 1`: two attempts of 250 ms exactly fill the 500 ms budget.

Watch out

[Counting attempts vs. retries] The Envoy worked example writes this fit as “ $2 \times 250 = 500\text{ms}$, equal to the timeout.” That 2 is the number of *attempts*, not the `num_retries` value — two attempts means the initial try plus *one* retry, i.e. `num_retries = 1`. The arithmetic is identical; only the label differs. Always pin down whether a config counts *retries* or *attempts* before trusting a budget calculation — off-by-one here silently doubles your worst-case latency.

4 Step 3 — The composed-timeout trap

Now set `num_retries = 3` (four attempts) with the same 250 ms per try — a plausible-looking but broken config.

Where the surplus attempts die

The worst case is

$$(\text{num_retries} + 1) \times \text{per_try} = (3 + 1) \times 250 = 1000 \text{ ms},$$

which overflows the 500 ms outer timeout by 500 ms. But the outer timeout fires at 500 ms no matter what, so the number of attempts that can *actually finish* is

$$\left\lfloor \frac{\text{outer}}{\text{per_try}} \right\rfloor = \left\lfloor \frac{500}{250} \right\rfloor = 2.$$

After two attempts each burn their full 250 ms, elapsed time is 500 ms and the remaining budget is $500 - 500 = 0$ ms. The third attempt is handed a 0 ms clock: it is killed at the outer timeout the instant it would begin. ✓

The third try is pure waste

With $\text{outer} = 500$ ms and $\text{per_try} = 250$ ms, the maximum *useful* retry count is

$$\left\lfloor \frac{\text{outer}}{\text{per_try}} \right\rfloor - 1 = \left\lfloor \frac{500}{250} \right\rfloor - 1 = 1.$$

Configuring $\text{num_retries} = 3$ does not buy three retries' worth of resilience — it buys *one*. The extra two are dead on arrival. The config *claims* a robustness it cannot deliver, and that gap is exactly the kind of thing that slips through code review.

5 Step 4 — Why stacked timeouts waste *compute*, not just time

A killed attempt is worse than a no-op. In the borderline case where the per-try timeout is slightly *under* the budget (say 200 ms with a 500 ms outer), a doomed later attempt actually *starts*: it seizes an engine slot, loads the KV cache, begins generating tokens, and burns real GPU cycles — right up until the outer timeout kills it mid-flight and returns nothing to the user.

Intuition

The engine is the scarce resource (Module 01: decode is memory-bandwidth-bound, one slot at a time). A retry that cannot possibly finish still *occupies* a slot that a *live* request could have used. So a misconfigured retry budget does not merely waste the retrying user's time — it steals throughput from every other user by parking doomed work on the GPU. Stacked timeouts convert a latency bug into a capacity bug.

Watch out

The fix is a single inequality, checked at config-review time:

$$(\text{num_retries} + 1) \times \text{per_try_timeout} \leq \text{outer_timeout}.$$

If it fails, either lower num_retries , shorten per_try_timeout , or raise the outer budget — but never ship a retry count the clock cannot honour. This is the composed-timeout trap, and it is easy to miss precisely because each number looks reasonable in isolation; only their *product* exposes the bug.

6 The argument, at a glance

#	Step	Result
1	Decompose	$50+30+20+350+50 = 500 \text{ ms} = B$; margin = 10%
2	Constraint	$(\text{retries}+1) \cdot \text{per_try} \leq \text{outer}$; fits at $\text{retries} \leq 1$
3	Trap	$\text{retries}=3 \Rightarrow 1000 \text{ ms}$ worst case; only 2 attempts land
4	Compute	doomed attempt seizes an engine slot, returns nothing

6.1 Equation cheat-sheet

Budget decomposition	$B = T_{\text{gw}} + T_{\text{auth}} + T_{\text{sched}} + T_{\text{engine}} + T_{\text{margin}}$
Margin rule	$\sum(\text{hard layers}) \leq 0.9 B$
Total attempts	$\text{num_retries} + 1$
Worst-case latency	$(\text{num_retries} + 1) \times \text{per_try_timeout}$
Fit constraint	$(\text{num_retries} + 1) \cdot \text{per_try} \leq \text{outer}$
Attempts that finish	$\lfloor \text{outer} / \text{per_try} \rfloor$
Max useful retries	$\lfloor \text{outer} / \text{per_try} \rfloor - 1$

Appendix A — Reference implementation

Every number above is produced by `m07_t04_timeout_budget_engine.py` (provided alongside this PDF). Run it to reproduce all figures; change B , the per-layer allocation, the per-try timeout, or `num_retries` to check your own policy.

```

1  """
2  Reference implementation for: Module 07 - Topic 4
3  "Composed Timeout Budgets & the Retry/Timeout Constraint."
4
5  Every number printed here is transcribed (rounded) into the worked-by-hand PDF,
6  so the arithmetic in the document is exact and self-consistent.
7
8  Scenario: a Nexus v7 interactive-class request with a 500 ms end-to-end budget.
9  The budget decomposes across gateway -> auth -> scheduler -> engine -> streaming
10 margin. Separately, retries compose with per-try timeouts; we show that with
11 2 retries x 250 ms the third try is pure waste against the 500 ms outer timeout.
12
13 Convention: all times in milliseconds.
14 """
15
16 # -----
17 # 0. The system under study.
18 # -----
19 B = 500.0    # end-to-end budget, ms
20
21 # Per-layer timeout allocation (ms). Sums to B with a streaming margin.
22 layers = {
23     "T_gateway":      50.0,
24     "T_auth":         30.0,
25     "T_scheduler":    20.0,
26     "T_engine":       350.0,
27     "T_streaming_margin": 50.0,
28 }
29
30 print("== 0. End-to-end budget ==")

```

```

31 print(f"B (end-to-end SLO) = {B:.0f} ms")
32
33 # -----
34 # 1. Budget decomposition: B = sum of per-layer timeouts.
35 # -----
36 print("\n== 1. Budget decomposition B = sum of per-layer timeouts ==")
37 total = 0.0
38 for name, t in layers.items():
39     print(f"    {name:20s} = {t:6.0f} ms")
40     total += t
41 print(f"    {'sum':20s} = {total:6.0f} ms")
42 print(f"check: sum == B ? {total} == {B} -> {'OK' if abs(total-B)<1e-9 else 'MISMATCH'}")
43
44 # The "leave 10% margin" guidance: hard layers (everything but the margin)
45 # should sit at <= 90% of B so there's slack for jitter.
46 hard_layers = total - layers['T_streaming_margin']
47 print(f"\nhard layers (excl. streaming margin) = {hard_layers:.0f} ms "
48       f"= {hard_layers/B*100:.0f}% of B")
49 print(f"streaming margin                = {layers['T_streaming_margin']:.0f} ms "
50       f"= {layers['T_streaming_margin']/B*100:.0f}% of B (the ~10% slack)")
51
52 # -----
53 # 2. The retry/timeout composition constraint.
54 #   total attempts = num_retries + 1 (the initial try plus the retries).
55 #   For all of them to fit: (num_retries + 1) * per_try_timeout <= outer.
56 # -----
57 print("\n== 2. Retry/timeout constraint ==")
58 per_try = 250.0      # per_try_timeout, ms
59 outer   = B          # outer (end-to-end) timeout, ms
60
61 def total_attempts(num_retries):      return num_retries + 1
62 def worst_case(num_retries, per_try): return total_attempts(num_retries) * per_try
63
64 print(f"per_try_timeout = {per_try:.0f} ms, outer_timeout = {outer:.0f} ms")
65 print(f"constraint: (num_retries + 1) * per_try_timeout <= outer_timeout")
66 print(f"{'num_retries':>12} {'attempts':>9} {'worst-case ms':>14} {'<= outer?':>12}")
67 for nr in [0, 1, 2, 3]:
68     wc = worst_case(nr, per_try)
69     ok = "fits" if wc <= outer else "OVERFLOW"
70     print(f"{nr:>12} {total_attempts(nr):>9} {wc:>14.0f} {ok:>12}")
71
72 # -----
73 # 3. The correct config: num_retries = 1 (initial + 1 retry = 2 attempts).
74 #   This is the largest num_retries whose worst case still fits 500 ms.
75 # -----
76 print("\n== 3. Correct config: num_retries = 1 (2 attempts) ==")
77 nr = 1
78 wc = worst_case(nr, per_try)
79 print(f"attempts = num_retries + 1 = {nr}+1 = {total_attempts(nr)}")
80 print(f"worst-case = {total_attempts(nr)} x {per_try:.0f} = {wc:.0f} ms "
81       f"= outer {outer:.0f} ms -> exactly fills the budget")
82 print(f"both attempts can complete inside {outer:.0f} ms in the worst case.")
83 print(f"NOTE: Envoy's worked example writes this as '2 x 250 = 500ms', i.e.")
84 print(f"      2 ATTEMPTS x 250 ms. Counting attempts, 2 is the ceiling.")
85
86 # -----
87 # 4. The trap: num_retries = 3 (4 attempts) with the same 250 ms per-try.
88 # -----
89 print("\n== 4. The composed-timeout trap: num_retries = 3 (4 attempts) ==")
90 nr = 3
91 attempts = total_attempts(nr)
92 wc = worst_case(nr, per_try)
93 overflow = wc - outer

```

```

94 print(f"attempts = {nr}+1 = {attempts}, worst-case = {attempts} x {per_try:.0f} "
95       f"= {wc:.0f} ms")
96 print(f"outer_timeout = {outer:.0f} ms -> worst-case overflow = {overflow:.0f} ms")
97 # The outer timeout fires at 500 ms; only floor(500/250)=2 attempts can finish.
98 attempts_that_fit = int(outer // per_try)
99 print(f"attempts that can actually finish = floor(outer/per_try) "
100       f"= floor({outer:.0f}/{per_try:.0f}) = {attempts_that_fit}")
101 print(f"> attempts 3 and 4 are killed at the {outer:.0f} ms outer timeout.")
102 # When the 3rd attempt would start, how much budget is left?
103 elapsed_before_3rd = 2 * per_try
104 remaining = outer - elapsed_before_3rd
105 print(f"after 2 attempts each burning {per_try:.0f} ms, elapsed = "
106       f"{elapsed_before_3rd:.0f} ms; remaining budget = {remaining:.0f} ms")
107 print(f"the 3rd attempt needs {per_try:.0f} ms but has only {remaining:.0f} ms "
108       f"-> killed before it can return: PURE WASTE.")
109
110 # -----
111 # 5. Wasted compute and the max useful num_retries.
112 # -----
113 print("\n== 5. Why stacked timeouts waste compute ==")
114 max_useful_retries = int(outer // per_try) - 1
115 print(f"max useful num_retries = floor(outer/per_try) - 1 "
116       f"= floor({outer:.0f}/{per_try:.0f}) - 1 = {max_useful_retries}")
117 print(f"> with per_try={per_try:.0f} ms and outer={outer:.0f} ms, at most "
118       f"{max_useful_retries} retry can ever land.")
119 print(f"with per_try slightly under the budget (e.g. 200 ms), a doomed later")
120 print(f"attempt DOES start and seizes an engine slot, burning GPU before the")
121 print(f"outer timeout kills it mid-flight and returns nothing. Configuring")
122 print(f"num_retries beyond floor(outer/per_try)-1 is a lie the math exposes.")

```

— END OF DOCUMENT —